

Spring Boot Starter

План доклада:

- Что такое Spring Boot Starter?
- Подключение Spring Boot Starter в проект с Maven и Gradle
- Правила наименования стартеров
- Структура Spring Boot Starter
- Кастомный Starter.





Spring Boot Starter – это набор наиболее удобных дескрипторов зависимостей, которые вы можете включить в свое приложение. Таким образом вы получаете универсальный магазин нужных вам технологий для реализации приложения на Spring без необходимости просматривать примеры кода и копировать-вставлять множество дескрипторов зависимостей. Что очень удобно.



Преимущества Spring Boot Starter перед обычными библиотеками.

- Возможность подключения одним дескриптором целого набора библиотек, необходимых для нашего направления разработки.
- Автоконфигурация создаваемых из стартера бинов (присвоение бинам наиболее часто используемых параметров, установка нужных связей между объектами и т.д.).
- Возможность ручной настройки создаваемых бинов через *application.properties*.



Недостатки Spring Boot Starter

- Единственным недостатком такого инструмента является то, что иногда starter не предоставляет разработчику нужного ему функционала. Однако эта проблема легко решается разработкой собственного starter.

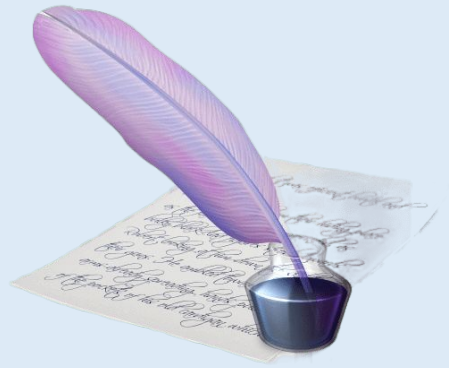
Подключение Spring Boot Starter к проекту

Maven

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter</artifactId>  
  <version>x.x.x</version>  
</dependency>
```

Gradle

```
dependencies {  
    implementation group: 'org.springframework.boot',  
        name: 'spring-boot-starter',  
        version: 'x.x.x'  
}
```



Правила наименования стартеров

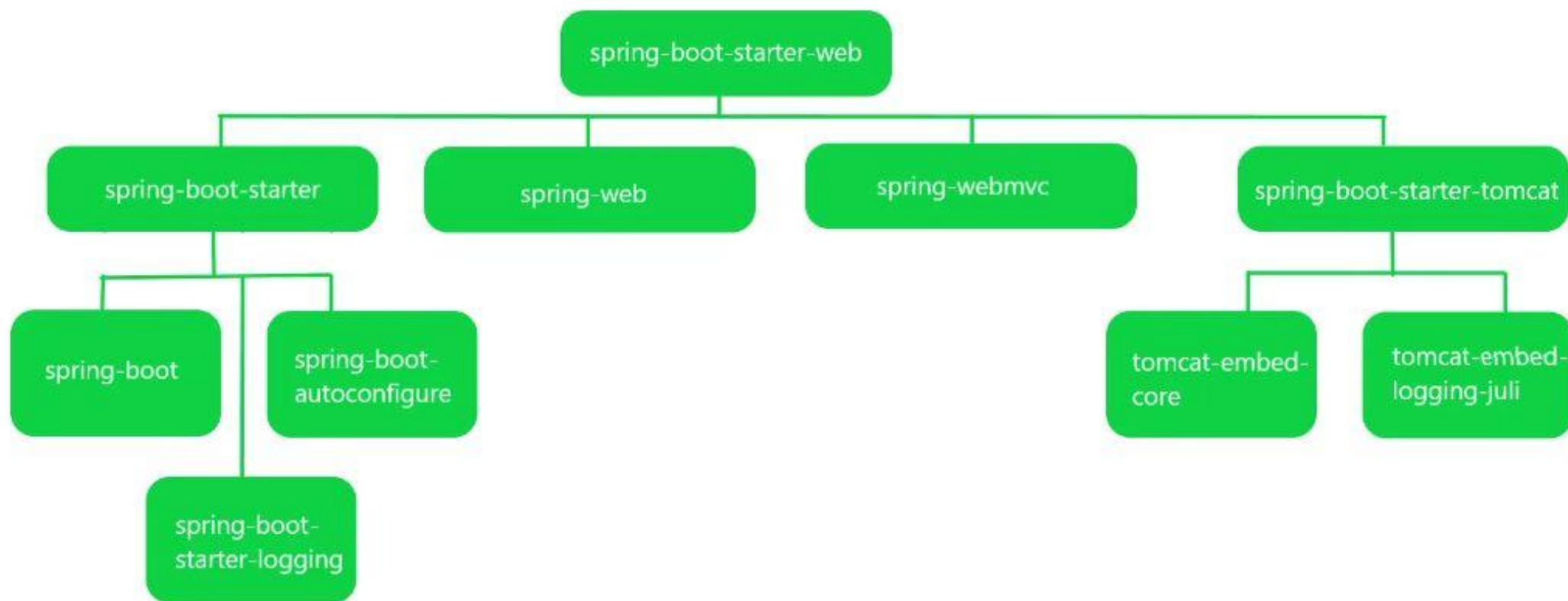
Схемы

наименования:

- “Официальные” стартеры – следуют схеме наименования `spring-boot-starter-*` , где * это конкретный тип приложения. Примеры: *spring-boot-starter-data-jpa*, *spring-boot-starter-data-mongodb*.
- “Сторонние” или “Пользовательские” стартеры – следуют схеме `*-spring-boot-starter`, где * это конкретный тип приложения. Пример: *thirdpartyproject-spring-boot-starter*.

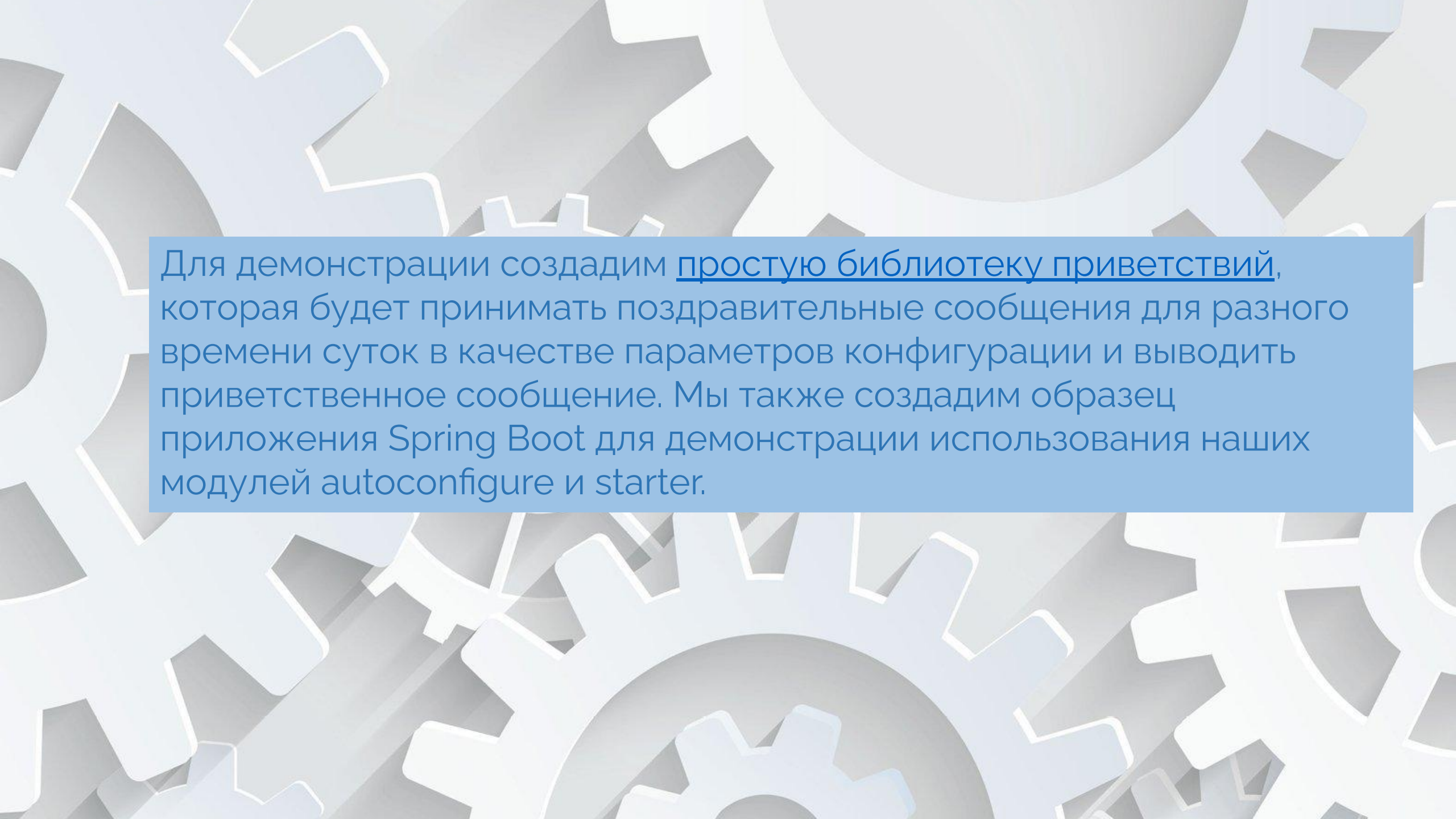


Структура Spring Boot Starter



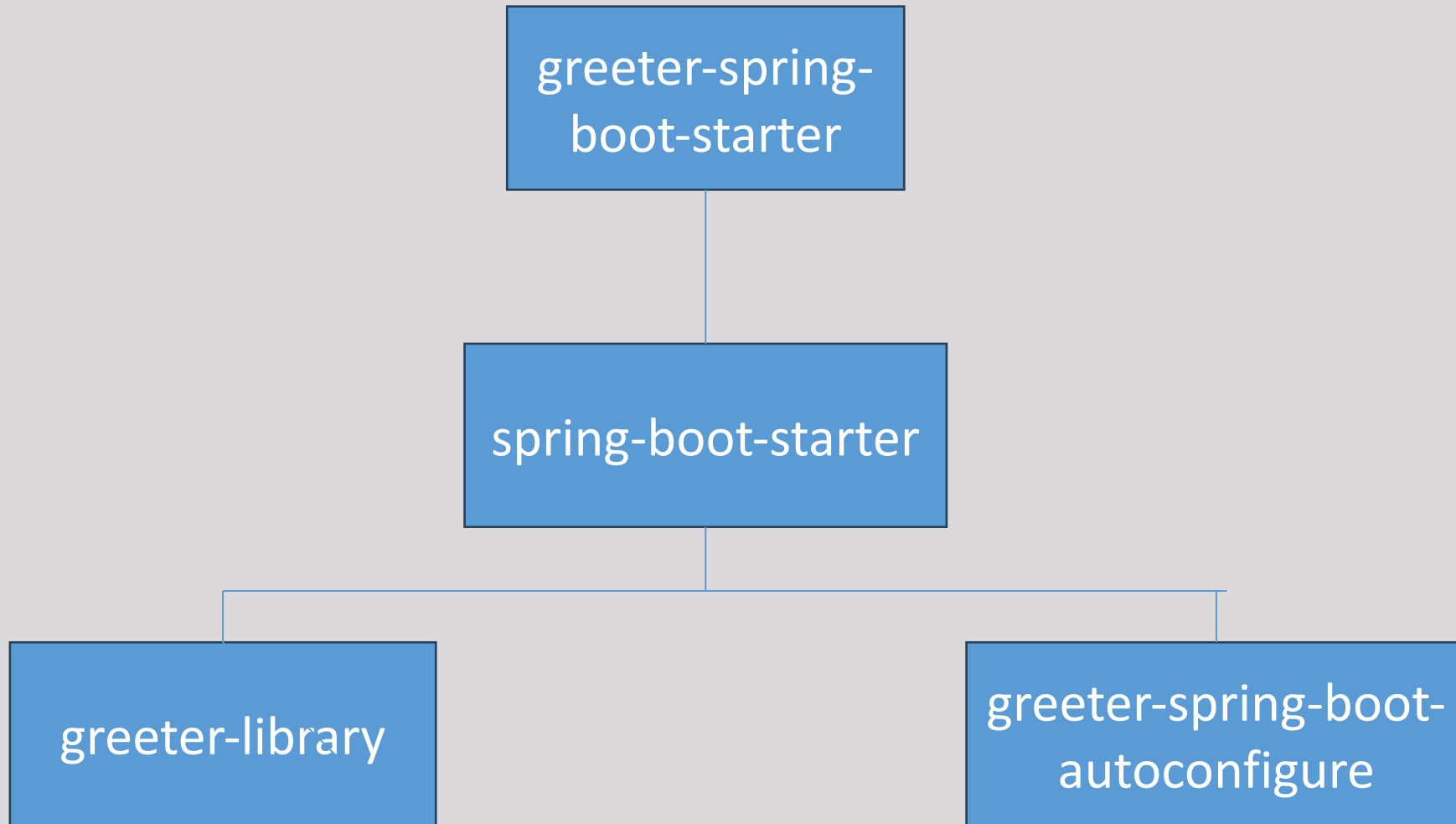
A black and white photograph of a propeller engine, likely from a vintage aircraft. The propeller is in the foreground, and the engine block is visible behind it. The text 'Заводим пользовательский starter' is overlaid on the right side of the image.

Заводим
пользовательский
starter

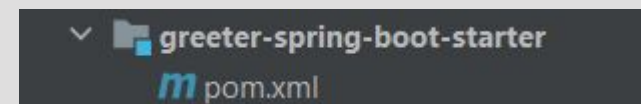
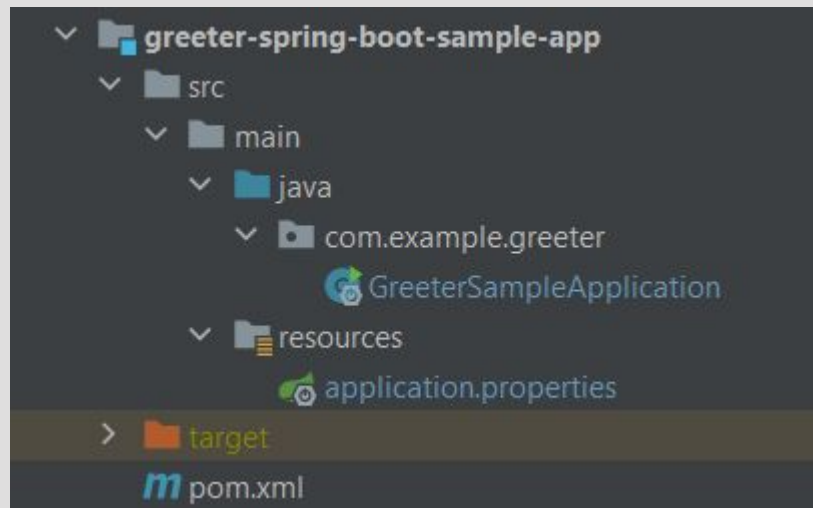
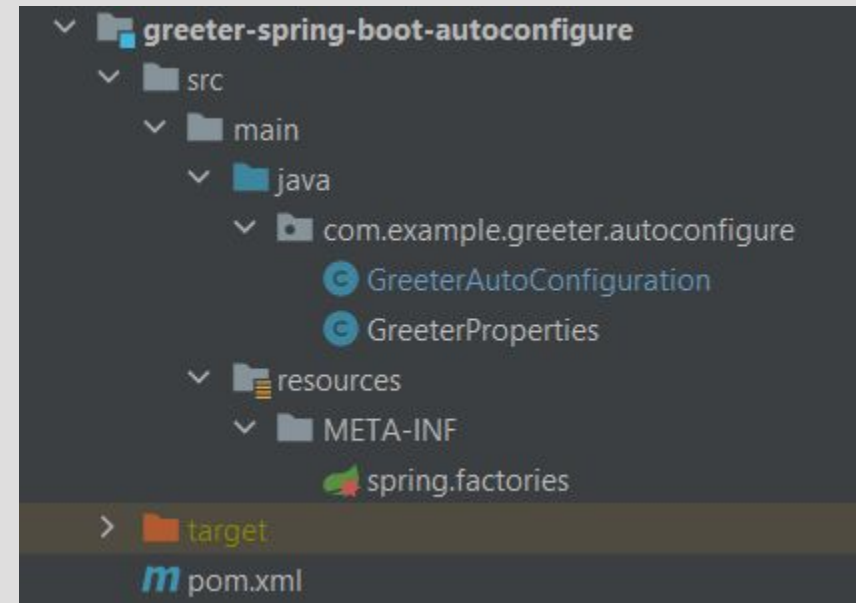
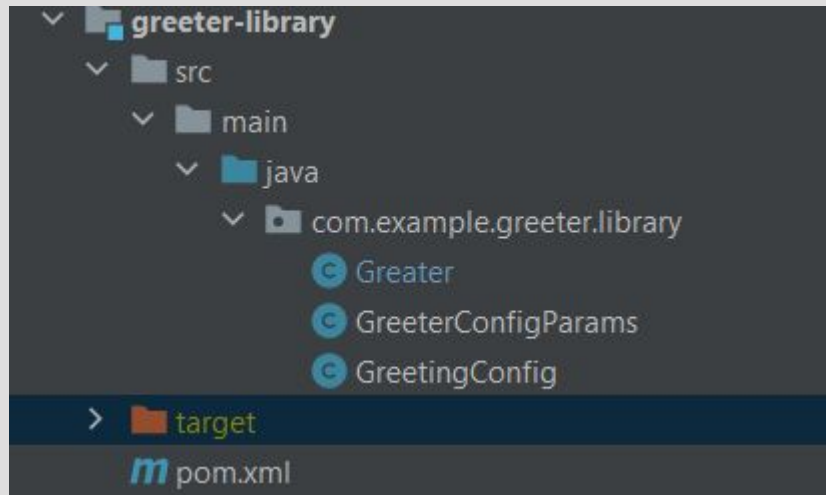


Для демонстрации создадим простую библиотеку приветствий, которая будет принимать поздравительные сообщения для разного времени суток в качестве параметров конфигурации и выводить приветственное сообщение. Мы также создадим образец приложения Spring Boot для демонстрации использования наших модулей `autoconfigure` и `starter`.

Структура custom starter



Создаем несколько модулей для starter



Модуль greeter-library

```
public class Greeter {  
    6 usages  
    private GreetingConfig greetingConfig;  
  
    1 usage  👤 Kashevar128 *  
    public Greeter(GreetingConfig greetingConfig) { this.greetingConfig = greetingConfig; }  
  
    1 usage  👤 Kashevar128  
    public String greet(LocalDateTime localDateTime) {  
  
        String name = greetingConfig.getProperty(USER_NAME);  
        int hourOfDay = localDateTime.getHour();  
  
        if (hourOfDay >= 5 && hourOfDay < 12) {  
            return String.format("Hello %s, %s", name, greetingConfig.get(MORNING_MESSAGE));  
        } else if (hourOfDay >= 12 && hourOfDay < 17) {  
            return String.format("Hello %s, %s", name, greetingConfig.get(AFTERNOON_MESSAGE));  
        } else if (hourOfDay >= 17 && hourOfDay < 20) {  
            return String.format("Hello %s, %s", name, greetingConfig.get(EVENING_MESSAGE));  
        } else {  
            return String.format("Hello %s, %s", name, greetingConfig.get(NIGHT_MESSAGE));  
        }  
    }  
  
    1 usage  👤 Kashevar128  
    public String greet() {  
        return greet(LocalDateTime.now());  
    }  
}
```

```
public class GreetingConfig extends Properties {  
  
}
```

```
public class GreeterConfigParams {  
    2 usages  
    public static final String USER_NAME = "user.name";  
    2 usages  
    public static final String MORNING_MESSAGE = "morning.message";  
    2 usages  
    public static final String AFTERNOON_MESSAGE = "afternoon.message";  
    2 usages  
    public static final String EVENING_MESSAGE = "evening.message";  
    2 usages  
    public static final String NIGHT_MESSAGE = "night.message";  
}
```


Модуль greeter-spring-boot-autoconfigure

```
@Configuration
@ConditionalOnClass(Greater.class)
@EnableConfigurationProperties(GreeterProperties.class)
public class GreeterAutoConfiguration {
```

10 usages

@Autowired

```
private GreeterProperties greeterProperties;
```

↑ Kashevar128

```
@Bean
```

```
@ConditionalOnMissingBean
```

```
public GreetingConfig greeterConfig() {
```

```
    String userName = greeterProperties.getUserName() == null ? System.getProperty("user.name") : greeterProperties.getUserName();
    String morningMessage = greeterProperties.getMorningMessage() == null ? "Good Morning" : greeterProperties.getMorningMessage();
    String afternoonMessage = greeterProperties.getAfternoonMessage() == null ? "Good Afternoon" : greeterProperties.getAfternoonMessage();
    String eveningMessage = greeterProperties.getEveningMessage() == null ? "Good Evening" : greeterProperties.getEveningMessage();
    String nightMessage = greeterProperties.getNightMessage() == null ? "Good Night" : greeterProperties.getNightMessage();
```

```
    GreetingConfig greetingConfig = new GreetingConfig();
    greetingConfig.put(USER_NAME, userName);
    greetingConfig.put(MORNING_MESSAGE, morningMessage);
    greetingConfig.put(AFTERNOON_MESSAGE, afternoonMessage);
    greetingConfig.put(EVENING_MESSAGE, eveningMessage);
    greetingConfig.put(NIGHT_MESSAGE, nightMessage);
    return greetingConfig;
```

```
}
```

↑ Kashevar128 *

```
@Bean
```

```
@ConditionalOnMissingBean
```

```
public Greater greeter(GreetingConfig greetingConfig) { return new Greater(greetingConfig); }
```


Модуль greeter-spring-boot-autoconfigure (продолжение)

```
@ConfigurationProperties(prefix = "example.greeter")
public class GreeterProperties {

    2 usages
    private String userName;

    2 usages
    private String morningMessage;

    2 usages
    private String afternoonMessage;

    2 usages
    private String eveningMessage;

    2 usages
    private String nightMessage;

    💡 //getters, setters
```

Модуль greeter-spring-boot-autoconfigure (продолжение)

```
@ConditionalOnClass(Greater.class)
```

- Включает инициализацию файла конфигурации при наличии класс *Greater* в *classpath*.

```
@Configuration
```

- Указывает на то, что данный класс является источником определений бинов.

```
@EnableConfigurationProperties(GreeterProperties.class)
```

- Осуществляет поддержку *@ConfigurationProperties*

```
@Bean
```

- Подсвечивает объекты, которые нужно создать *Spring*.

Модуль greeter-spring-boot-autoconfigure (продолжение)

`@ConditionalOnMissingBean`

- *Устанавливает такой приоритет, что пользовательские переопределенные бины создаются в первую очередь.*

`@Autowired`

- *Позволяет создать нужный бин и положить в объявленную переменную.*

`@ConfigurationProperties(prefix = "example.greeter")`

- *Аннотация для работы со свойствами.*

Модуль greeter-spring-boot-autoconfigure (продолжение)



```
org.springframework.boot.autoconfigure.EnableAutoConfiguration=com.example.greeter.autoconfigure.GreeterAutoConfiguration
```

- *Позволяет starter обнаружить нужную автоконфигурацию*

Принцип действия

модуля:

При запуске приложения класс *GreeterAutoConfiguration* будет запущен, если класс *Greeter* присутствует в classpath. В случае успешного запуска он заполнит контекст приложения Spring с помощью компонентов *GreeterConfig* и *Greeter*, прочитав свойства через класс *GreeterProperties*.

[Аннотация @ConditionalOnMissingBean](#) гарантирует, что эти компоненты будут созданы только в том случае, если они еще не существуют. Это позволяет разработчикам полностью переопределять автоматически настраиваемые компоненты, определяя свои собственные в одном из классов *@Configuration*.

Модуль greeter-spring-boot-autoconfigure (продолжение)

Файл pom

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
  </dependency>
  <dependency>
    <groupId>org.yaml</groupId>
    <artifactId>snakeyaml</artifactId>
    <version>2.0</version>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-configuration-processor</artifactId>
    <optional>true</optional>
  </dependency>
  <dependency>
    <groupId>com.example</groupId>
    <artifactId>greeter-library</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <scope>compile</scope>
  </dependency>
</dependencies>
```


Модуль greeter-spring-boot-starter

Файл pom

```
<artifactId>greeter-spring-boot-starter</artifactId>
<packaging>jar</packaging>

<dependencies>
  <dependency>
    <groupId>com.example</groupId>
    <artifactId>greeter-spring-boot-autoconfigure</artifactId>
    <version>0.0.1-SNAPSHOT</version>
  </dependency>
  <dependency>
    <groupId>com.example</groupId>
    <artifactId>greeter-library</artifactId>
    <version>0.0.1-SNAPSHOT</version>
  </dependency>
</dependencies>
```

Модуль greeter-spring-boot-sample-app

```
@SpringBootApplication
public class GreeterSampleApplication implements CommandLineRunner {

    1 usage
    @Autowired
    private Greeter greeter;

    Kashevar128
    public static void main(String[] args) { SpringApplication.run(GreeterSampleApplication.class, args); }

    Kashevar128 *
    @Override
    public void run(String... args) throws Exception {
        LocalDateTime localDateTime = LocalDateTime.of(year: 2023, Month.MAY, dayOfMonth: 19, hour: 11, minute: 5, second: 15);
        String message = greeter.greet();
        //String message = greeter.greet(localDateTime);
        System.out.println(message);
    }
}
```

resources
application.properties

```
example.greeter.userName=Bro
example.greeter.eveningMessage=Hi!
server.port=8085
```

- Здесь можно установить пользовательские настройки, они будут в приоритете перед автоконфигурацией

Модуль greeter-spring-boot-sample-app(продолжение)

Файл pom

```
<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
  <dependency>
    <groupId>com.example</groupId>
    <artifactId>greeter-spring-boot-starter</artifactId>
    <version>0.0.1-SNAPSHOT</version>
  </dependency>
</dependencies>
```

Spring Boot автоматически все настроит, и у нас будет компонент *Greeter*, готовый к внедрению и использованию.

Заключение

Для создания пользовательского стартера нам необходимо написать следующие компоненты:

- Класс автоматической настройки для нашей библиотеки вместе с классом свойств для пользовательской настройки.
- Стартовый *port* для ввода зависимостей библиотеки и проекта `autoconfigure`.
- Не забываем прописывать в `spring.factories` реализацию `EnableAutoConfiguration`.
- Не забываем про аннотации `@ConditionalOnClass`, не забываем про аннотацию `@EnableConfigurationProperties`.

здесь хорошо описан кастомный стартер <https://www.baeldung.com/spring-boot-custom-starter>

GitHub - Kashevar128/spring-boot-custom-starter
<https://github.com>